



Retail Demand Forecasting

Name - Ishika

Group - 3

Emp ID - 7145

1. Create Storage Account and container namely:

Storage Account - ishikastorage

Conatiner name - capstoneconatiner

The screenshot shows the Microsoft Azure portal interface for a storage account named 'capstonecontainer'. The container is currently empty. The table below shows the contents of the container, which are three folders: '__unitystorage', 'bronze', and 'landing'. The table columns are Name, Last modified, Access tier, Blob type, Size, and Lease state.

Name	Last modified	Access tier	Blob type	Size	Lease state
__unitystorage	8/30/2025, 4:23:46 PM				...
bronze	8/30/2025, 5:51:11 PM				...
landing	8/30/2025, 3:47:54 PM				...

Dropped raw csv files in the folders.

capstonecontainer - Microsoft Az...Capstone-bronze+silver - Databri...gold - DatabricksUntitled document - Google Docs

https://portal.azure.com/#view/Microsoft_Azure_Storage/ContainerMenuBlade/~-/overview/storageAccountId/%2Fsubscriptions%2Fd008c7cc-979...

Microsoft AzureSearch resources, services, and docs (G+)

Copilot

tredecence9@michaelklou...DEFAULT DIRECTORY (MICHAELK...

Home >

capstonecontainerContainer

Search

+ Add Directory ↑ Upload ↻ Refresh 🗑 Delete 📄 Copy 📄 Paste 🔄 Rename ⚙ Acquire lease ⚙ Break lease 🛠 Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

capstonecontainer > bronze

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)Only show active objects

Showing all 8 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[-]					...
☐	_checkpoints	8/30/2025, 5:52:30 PM				...
☐	_schemas	8/30/2025, 5:51:11 PM				...
☐	holidays	8/30/2025, 5:59:58 PM				...
☐	inventory	8/30/2025, 5:59:45 PM				...
☐	pos	8/30/2025, 5:52:35 PM				...
☐	products	8/30/2025, 6:01:06 PM				...
☐	stores	8/30/2025, 5:59:35 PM				...
☐	weather	8/30/2025, 6:00:12 PM				...

Add or remove favorites by pressing Ctrl+Shift+F

27°C Mostly cloudy

Search

4:59 PM 8/31/2025

2. Create Unity Catalog names ishi_catalog

ishi_catalog

Share

Create schema

Overview

Details

Permissions

Workspaces

Description

AI generate

Add

Filter schemas

6 schemas

Name	Owner	Created at
bronze	tredence9@michaelkloudkraft...	Aug 30, 2025, 04:17 PM
default	tredence9@michaelkloudkraft...	Aug 30, 2025, 04:04 PM
gold	tredence9@michaelkloudkraft...	Aug 30, 2025, 04:17 PM
information_schema	System user	Aug 30, 2025, 04:04 PM
landing	tredence9@michaelkloudkraft...	Aug 30, 2025, 04:52 PM
silver	tredence9@michaelkloudkraft...	Aug 30, 2025, 04:17 PM

About this catalog

Owner Tredence LabsKraft9

- Formed Groups :
- 1. Data analysts
 - 2. Data Scientists
 - 3. Data Engineers Ishika

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

tred_d3

+ New

- Workspace
- Recents
- Catalog
- Jobs & Pipelines
- Compute
- Marketplace
- SQL
 - SQL Editor
 - Queries
 - Dashboards
 - Genie
 - Alerts
 - Query History
 - SQL Warehouses
- Data Engineering
 - Job Runs
 - Data Ingestion
- AI/ML

Settings

- Workspace admin
- Appearance
- Identity and access
- Security
- Compute
- Development
- Notifications
- Advanced
- User
 - Profile
 - Preferences
 - Developer
 - Linked accounts
 - Notifications

Workspace settings > Identity and access >

Groups

Filter groups 5 total

Add group

Name	Members	Source
Data Analysts	2	Account ⓘ
Data Engineers ishika	1	Account ⓘ
Data Scientists	2	Account ⓘ
admins	7	System ⓘ
users	11	System ⓘ

< Previous Next > 20 / page

Given RBAC Permission on groups in unity catalog (ishi_catalog)

Grant on ishi_catalog

Granted privileges will be inherited by applicable objects (e.g. schemas, tables) in this catalog. [Learn more](#)

Principals

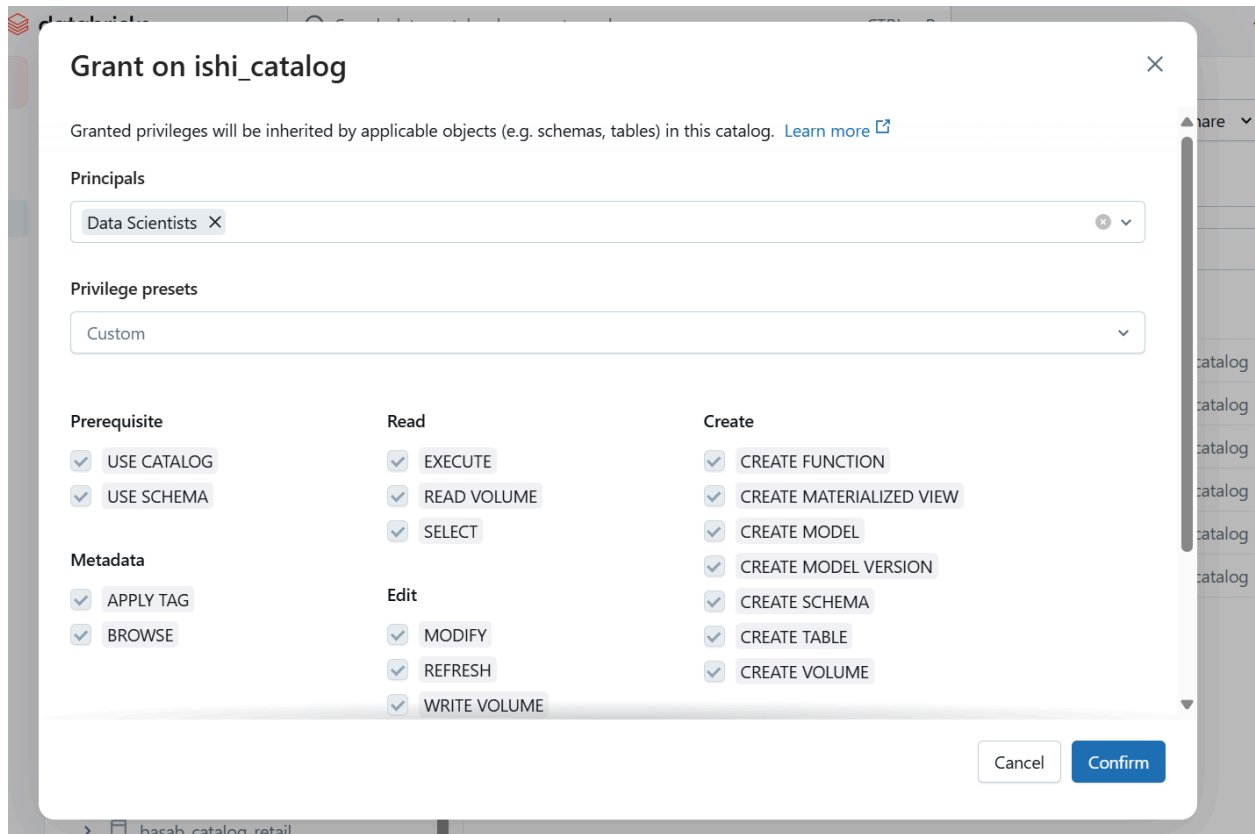
Data Engineers ishika X

Privilege presets

Custom

Prerequisite	Read	Create
☒ USE CATALOG	☒ EXECUTE	☐ CREATE FUNCTION
☒ USE SCHEMA	☒ READ VOLUME	☐ CREATE MATERIALIZED VIEW
	☒ SELECT	☐ CREATE MODEL
Metadata	Edit	☐ CREATE MODEL VERSION
☐ APPLY TAG	☒ MODIFY	☐ CREATE SCHEMA
☐ BROWSE	☐ REFRESH	☐ CREATE TABLE
	☐ WRITE VOLUME	☐ CREATE VOLUME

Cancel Confirm



3. Raw Data Ingestion (Bronze Layer)

- Connected to **ADLS Gen2 / DBFS** for raw file ingestion.
- Ingested datasets:
 - **POS Transactions** (pos_raw)
 - **Stores Metadata** (stores_raw)
 - **Products Metadata** (products_raw)
 - **Holidays Data** (holidays_raw)
 - **Weather Data** (weather_raw)
 - **Inventory Data** (inventory_raw)

Ex: for POS file

Ingestion code:

```
from pyspark.sql.functions import col

import re

# Helper function to clean column names

def clean_column_names(df):

    for c in df.columns:

        new_c = re.sub(r'^a-zA-Z0-9_', '_', c) # Replace invalid chars with _

        df = df.withColumnRenamed(c, new_c)

    return df

# Define paths

pos_source =
"abfss://capstonecontainer@ishikastorage.dfs.core.windows.net/landing/pos"

schema_loc =
"abfss://capstonecontainer@ishikastorage.dfs.core.windows.net/bronze/_schemas/p
os"

chkpt      =
"abfss://capstonecontainer@ishikastorage.dfs.core.windows.net/bronze/_checkpoin
ts/pos"

bronze_pos =
"abfss://capstonecontainer@ishikastorage.dfs.core.windows.net/bronze/pos"      #
<- this was missing earlier

# Auto Loader stream

pos_df = (spark.readStream

    .format("cloudFiles")

    .option("cloudFiles.format", "csv")
```

```

.option("header", "true")

.option("inferSchema", "true")

.option("cloudFiles.schemaLocation", schema_loc)

.load(pos_source)
)

# Clean column names
pos_df = clean_column_names(pos_df)

# Write to Bronze Delta
(pos_df.writeStream
  .format("delta")
  .option("checkpointLocation", chkpt)
  .option("path", bronze_pos)
  .trigger(once=True)    # treat like batch
  .outputMode("append")
  .start())

```

```

### md

### **Registering the Delta paths as managed tables in Unity Catalog
(ishi_catalog)**

```

```

%sql

CREATE TABLE IF NOT EXISTS ishi_catalog.bronze.pos

```

```
USING DELTA

LOCATION
'abfss://capstonecontainer@ishikastorage.dfs.core.windows.net/bronze/pos';
```

- Data loaded in streaming/batch mode using:

```
pos_raw = (spark.readStream

.format("cloudFiles")

.option("cloudFiles.format", "csv")

.load("abfss://container/raw/pos"))
```

4. Data Cleaning & Transformation (Silver Layer)

- Applied schema definitions and transformations.
- Standardized column formats:
 - Trimmed strings
 - Converted dates to date type
 - Converted product names to uppercase
- Created curated **Silver tables**:
 - ishi_catalog.silver.pos
 - ishi_catalog.silver.stores
 - ishi_catalog.silver.products

- ishi_catalog.silver.holidays
- ishi_catalog.silver.weather
- ishi_catalog.silver.inventory

Example cleaning:

For pos table :

```
from pyspark.sql.functions import col, when, trim

from pyspark.sql.types import IntegerType, DoubleType, DateType

import pyspark.sql.functions as F

# Read Bronze POS table

pos_df = spark.table("ishi_catalog.bronze.pos")

# Clean POS data

pos_silver_df = (

    pos_df

    # Trim string fields

    .withColumn("store_id", trim(col("store_id")))

    .withColumn("sku_id", trim(col("sku_id")))

    # Remove invalid IDs

    .filter(~col("store_id").rlike("INVALID|NULL") &

col("store_id").isNotNull())

    .filter(~col("sku_id").rlike("INVALID|NULL") & col("sku_id").isNotNull())

    # Date cleansing
```

```

.withColumn("date", F.to_date("date", "yyyy-MM-dd"))

.filter(col("date").isNotNull()) # drop invalid dates

# Units sold cleanup (convert to int, drop negatives/nulls)

.withColumn("units_sold", col("units_sold").cast(IntegerType()))

.filter(col("units_sold").isNotNull() & (col("units_sold") >= 0))

# Price cleanup (convert to double, drop nulls/negatives/outliers)

.withColumn("price", col("price").cast(DoubleType()))

.filter(col("price").isNotNull() & (col("price") > 0) & (col("price") <
1000))

# Promo flag cleanup (cast to int, keep only 0/1, else set to 0)

.withColumn("promo_flag",
            when(col("promo_flag").isin("1", "0"),
col("promo_flag").cast(IntegerType()))

            .otherwise(F.lit(0)))

.drop("_rescued_data")
)

# Write Silver POS table
(
    pos_silver_df.write.format("delta")

    .mode("overwrite")

    .option("overwriteSchema", "true")

    .saveAsTable("ishi_catalog.silver.pos")
)

```

```
)  
  
print("Silver POS table created successfully!")
```

```
from pyspark.sql.functions import col, trim, upper  
  
products_silver = products_raw \  
    .withColumn("sku_id", trim(col("sku_id"))) \  
    .withColumn("product_name", upper(trim(col("product_name"))))
```

5. Gold Layer Design

Purpose of the **Gold layer**:

- **Enriched Fact tables** for analytics
- **Dimension tables** for lookup
- **Aggregated tables** (Daily/Weekly/Monthly) for reporting

5.1. Fact Tables

Sales Fact (sales_fact)

Combines POS with store, product, holidays, and weather.

Added derived column:

```
pos = pos.withColumn("revenue", col("price") * col("units_sold"))
```

```
sales_fact = (  
    pos
```

```

.join(stores, "store_id", "left")

.join(products, "sku_id", "left")

.join(holidays, "date", "left")

.join(weather, ["region", "date"], "left")

.withColumn("event_name", coalesce(col("event_name"), lit("NO_HOLIDAY")))

.withColumn("temperature_c", coalesce(col("temperature_c"), lit(0.0)))

.withColumn("rainfall_mm", coalesce(col("rainfall_mm"), lit(0.0)))

)

(sales_fact.write.format("delta")

.mode("overwrite")

.option("overwriteSchema", "true")

.saveAsTable("ishi_catalog.gold.sales_fact"))

print("Gold Sales Fact table created!")

```

-
- **Inventory Fact (inventory_fact)**
Combines inventory with product & store metadata.

```

#%%

inventory_fact = (

    inventory

    .join(stores, "store_id", "left")

    .join(products, "sku_id", "left")

)

```

```
(inventory_fact.write.format("delta")  
  
.mode("overwrite")  
  
.option("overwriteSchema", "true")  
  
.saveAsTable("ishi_catalog.gold.inventory_fact"))  
  
print("Gold Inventory Fact table created!")
```

5.2. Dimension Tables

- product_dim – product master details
- store_dim – store master details

```
products.write.format("delta") \  
  
.mode("overwrite") \  
  
.option("overwriteSchema", "true") \  
  
.saveAsTable("ishi_catalog.gold.product_dim")  
  
stores.write.format("delta") \  
  
.mode("overwrite") \  
  
.option("overwriteSchema", "true") \  
  
.saveAsTable("ishi_catalog.gold.store_dim")  
  
print("Gold Dimension tables created!")
```

5.3. Aggregated Tables

- daily_sales - by date, store, sku, region
- weekly_sales - by year, week, store, sku, region
- monthly_sales - by year, month, store, sku, region

Example aggregation:

```
daily_sales = (
    sales_fact.groupBy("date", "store_id", "sku_id", "region", "category")
    .agg(
        F.sum("units_sold").alias("total_units_sold"),
        F.sum("revenue").alias("total_revenue"),
        F.sum("promo_flag").alias("promo_count")
    )
)

daily_sales.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("ishi_catalog.gold.daily_sales")

# Weekly

weekly_sales = (
    sales_fact
    .withColumn("week", weekofyear(col("date")))
    .withColumn("year", year(col("date")))
    .groupBy("year", "week", "store_id", "sku_id", "region", "category")
```

```

        .agg(
            F.sum("units_sold").alias("total_units_sold"),
            F.sum("revenue").alias("total_revenue"),
            F.sum("promo_flag").alias("promo_count")
        )
    )

weekly_sales.write.format("delta") \

    .mode("overwrite") \

    .option("overwriteSchema", "true") \

    .saveAsTable("ishi_catalog.gold.weekly_sales")

# Monthly

monthly_sales = (

    sales_fact

    .withColumn("month", month(col("date")))

    .withColumn("year", year(col("date")))

    .groupBy("year", "month", "store_id", "sku_id", "region", "category")

    .agg(
        F.sum("units_sold").alias("total_units_sold"),
        F.sum("revenue").alias("total_revenue"),
        F.sum("promo_flag").alias("promo_count")
    )
)

monthly_sales.write.format("delta") \

    .mode("overwrite") \

    .option("overwriteSchema", "true") \

```

```
.saveAsTable("ishi_catalog.gold.monthly_sales")

print("Aggregated Daily / Weekly / Monthly Sales created successfully!")
```

```
daily_sales = sales_fact.groupBy("date", "store_id", "sku_id") \

    .agg(

        F.sum("units_sold").alias("total_units_sold"),

        F.sum("revenue").alias("total_revenue")

    )
```

6. Delta Live Tables (DLT) Pipeline for Gold

- Converted Gold transformations into a **DLT pipeline notebook**.
- Steps:
 1. Created a new notebook (DLT_Retail_Gold)
 2. Defined DLT tables using decorators `@dlt.table`
 3. Loaded Silver tables → joined them → produced Gold fact/dim/aggregate tables
 4. Configured DLT pipeline:
 - Source: this notebook
 - Target schema: ishi_catalog.gold

■ Mode: Continuous or Trigger Once

Example:

```
@dlt.table(name="sales_fact", comment="Enriched sales fact table")

def sales_fact():

    pos = dlt.read("silver_pos")

    stores = dlt.read("silver_stores")

    products = dlt.read("silver_products")

    holidays = dlt.read("silver_holidays")

    weather = dlt.read("silver_weather")


    return (

        pos.withColumn("revenue", col("price") * col("units_sold"))

        .join(stores, "store_id", "left")

        .join(products, "sku_id", "left")

        .join(holidays, "date", "left")

        .join(weather, ["region", "date"], "left")

    )
```

7. Next Steps

- Validate Gold tables (SELECT * FROM gold.sales_fact LIMIT 10).
- Expose Gold layer to BI tools (Power BI / Tableau).

Visualization in Power BI

